Experiment 5: Integrate ML Predictions in Frontend Using Fetch/AJAX

Aim

To create an interactive HTML frontend that communicates with the Flask ML API using JavaScript fetch API to display real-time predictions.

Software Requirements

- Python 3.10+
- Flask
- Previously created ML API from Experiment 4
- Modern web browser

Theory

AJAX (Asynchronous JavaScript and XML) allows web pages to update content dynamically without refreshing the entire page. The Fetch API is a modern JavaScript interface for making HTTP requests. By combining HTML forms, JavaScript event handlers, and fetch requests, we create an interactive user interface that communicates with our ML backend to provide instant predictions.

Procedure

Step 1: Create Project Directory

```
mkdir exp5_ml_frontend cd
exp5_ml_frontend
```

Step 2: Copy Files from Experiment 4

```
cp -r ../exp4_ml_api/* .
```

Step 3: Activate Virtual Environment

Windows:

```
venv\Scripts\activate
```

Mac/Linux:

```
source venv/bin/activate
```

Step 4: Create Project Structure

```
mkdir static mkdir
static/js
```

Step 5: Update app.py

```
from flask import Flask, request, jsonify, render_template
import pickle
import numpy as np

app = Flask(__name__

) # Load models
with open('iris_logistic_regression.pkl', 'rb') as f: model
    = pickle.load(f)

with open('feature_names.pkl', 'rb') as f:
    feature_names = pickle.load(f)

with open('target_names.pkl', 'rb') as f:
    target_names = pickle.load(f)

@app.route('/')
def home():
    return render_template('predict.html',
                           feature_names=feature_names,
                           target_names=target_names)

@app.route('/api/predict', methods=['POST']) def
predict():
    try:
        data = request.get_json() features
        = data.get('features')

        if not features or len(features) != 4:
            return jsonify({'error': 'Expected 4 features'}), 400

        features_array = np.array([features])

        # Predict
        prediction = model.predict(features_array)
        prediction_index = int(prediction[0]) predicted_class
        = target_names[prediction_index]

        # Get probabilities
        if hasattr(model, 'predict_proba'):
```

```
            probabilities = model.predict_proba(features_array)[0]
            prob_dict = {
                target_names[i]: float(probabilities[i])
                for i in range(len(target_names))
            }

        else:
            prob_dict = None

        response = {
            'prediction': predicted_class, 'prediction_index':
            prediction_index, 'input_features': {
                feature_names[i]: features[i] for i in
                range(len(features))
            },
            'probabilities': prob_dict
        }

        return jsonify(response) except Exception

    as e:
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Step 6: Create templates/predict.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Iris Flower Predictor</title>
</head>
<body>
    <div class="container">
        <h1>□ Iris Flower Species Predictor</h1>
        <p class="subtitle">Enter flower measurements to predict the species</p>

        <div class="form-container">
            <form id="predictionForm">
                <div class="form-row">
                    <div class="form-group">
                        <label for="sepal_length">Sepal Length (cm)</label>
                        <input type="number" id="sepal_length" step="0.1" min="0"
```

```
                    required placeholder="e.g., 5.1">
                <span class="hint">Typical range: 4.3 - 7.9</span>
            </div>

            <div class="form-group">
                <label for="sepal_width">Sepal Width (cm)</label>
                <input type="number" id="sepal_width" step="0.1" min="0"
                    required placeholder="e.g., 3.5">
                <span class="hint">Typical range: 2.0 - 4.4</span>
            </div>
        </div>


        <div class="form-row">
            <div class="form-group">
                <label for="petal_length">Petal Length (cm)</label>
                <input type="number" id="petal_length" step="0.1" min="0" required
                    placeholder="e.g., 1.4">
                <span class="hint">Typical range: 1.0 - 6.9</span>
            </div>

            <div class="form-group">
                <label for="petal_width">Petal Width (cm)</label>
                <input type="number" id="petal_width" step="0.1" min="0"
                    required placeholder="e.g., 0.2">
                <span class="hint">Typical range: 0.1 - 2.5</span>
            </div>
        </div>

        <button type="submit" class="btn-predict" id="predictBtn">
            <span id="btnText">Predict Species</span>
            <span id="btnLoader" class="loader" style="display: none;"></span>
        </button>
    </form>

    <div class="examples">
        <h3>Quick Examples:</h3>
        <button class="btn-example" onclick="fillExample(1)">Setosa Example</button>
        <button class="btn-example" onclick="fillExample(2)">Versicolor
Example</button>
        <button class="btn-example" onclick="fillExample(3)">Virginica
Example</button>
```

```
            </div>
        </div>

        <div id="result" class="result" style="display: none;">
            <h2>Prediction Result</h2>
            <div class="prediction-card">
                <div class="prediction-main">
                    <span class="label">Predicted Species:</span>
                    <span class="value" id="predictedClass"></span>
                </div>
            </div>

            <div id="probabilities" class="probabilities">
                <h3>Prediction Confidence</h3>
                <div id="probBars"></div>
            </div>

            <div class="input-summary">
                <h3>Input Values</h3>
                <div id="inputValues"></div>
            </div>
        </div>

        <div id="error" class="error" style="display: none;"></div>


    </div>

    <script src="{{ url_for('static', filename='js/predict.js') }}"></script>
</body>
</html>
```

Step 7: Create static/js/predict.js

```
// Example datasets
const examples = {
    1: { // Setosa
        sepal_length: 5.1,
        sepal_width: 3.5,
        petal_length: 1.4,
        petal_width: 0.2
    },
    2: { // Versicolor
        sepal_length: 5.7,
```

```
            sepal_width: 2.8,
            petal_length: 4.1,
            petal_width: 1.3
        },
        3: { // Virginica
            sepal_length: 6.7,
            sepal_width: 3.0,
            petal_length: 5.2,
            petal_width: 2.3
        }
};

// Fill form with example data function
fillExample(exampleNum) {
    const example = examples[exampleNum];
    document.getElementById('sepal_length').value = example.sepal_length;
    document.getElementById('sepal_width').value =
    example.sepal_width; document.getElementById('petal_length').value =
    example.petal_length; document.getElementById('petal_width').value =
    example.petal_width;
}

// Handle form submission
document.getElementById('predictionForm').addEventListener('submit', async function(e) {
    e.preventDefault();

    // Hide previous results/errors
    document.getElementById('result').style.display = 'none';
    document.getElementById('error').style.display = 'none';

    // Show loading state
    const predictBtn = document.getElementById('predictBtn');
    const btnText = document.getElementById('btnText');
    const btnLoader = document.getElementById('btnLoader');
    predictBtn.disabled = true;
    btnText.style.display = 'none';



    btnLoader.style.display = 'inline-block';

    // Get form values
```

```
    const features = [
        parseFloat(document.getElementById('sepal_length').value),
        parseFloat(document.getElementById('sepal_width').value),
        parseFloat(document.getElementById('petal_length').value),
        parseFloat(document.getElementById('petal_width').value)
    ];

    try {
        // Make API request
        const response = await fetch('/api/predict', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({features: features})
        });

        const data = await response.json(); if

        (response.ok) {
            displayResult(data);
        } else {
            displayError(data.error || 'An error occurred');
        }
    } catch (error) {
        displayError('Failed to connect to server: ' + error.message);
    } finally {
        // Reset button state
        predictBtn.disabled = false;
        btnText.style.display = 'inline';
        btnLoader.style.display = 'none';
    }
});

// Display prediction result
function displayResult(data) {
    // Show result container
    const resultDiv = document.getElementById('result');
    resultDiv.style.display = 'block';

    // Update predicted class
    document.getElementById('predictedClass').textContent = data.prediction;
```

```
// Display probabilities if available if
(data.probabilities) {
    const probBars = document.getElementById('probBars');
    probBars.innerHTML = '';

    for (const [className, probability] of Object.entries(data.probabilities)) { const
        percentage = (probability * 100).toFixed(2);


        const probBar = document.createElement('div');
        probBar.className = 'prob-bar';
        probBar.innerHTML = `
            <div class="prob-label">
                <span class="prob-name">${className}</span>
                <span class="prob-value">${percentage}%</span>
            </div>
            <div class="bar-container">
                <div class="bar-fill" style="width: ${percentage}%"></div>
            </div>
        `;

        probBars.appendChild(probBar);
    }
}

// Display input values
const inputValues = document.getElementById('inputValues');
inputValues.innerHTML = '';

for (const [featureName, value] of Object.entries(data.input_features)) { const
    inputItem = document.createElement('div');
    inputItem.className = 'input-item';
    inputItem.innerHTML = `
        <span>${featureName}:</span>
        <strong>${value} cm</strong>
    `;
    inputValues.appendChild(inputItem);
}

// Scroll to result
resultDiv.scrollIntoView({ behavior: 'smooth', block: 'nearest' });
}
```

```
// Display error message function
displayError(message) {
    const errorDiv = document.getElementById('error');
    errorDiv.textContent = 'Error: ' + message; errorDiv.style.display
    = 'block';
    errorDiv.scrollIntoView({ behavior: 'smooth', block: 'nearest' });
}
```

Step 8: Run the Application

```
python app.py
```

Step 9: Test the Application

1. Open browser at `http://localhost:5000`
2. Try the quick example buttons
3. Submit the form and watch the real-time prediction
4. Try different models and inputs
5. Observe the probability bars and confidence scores

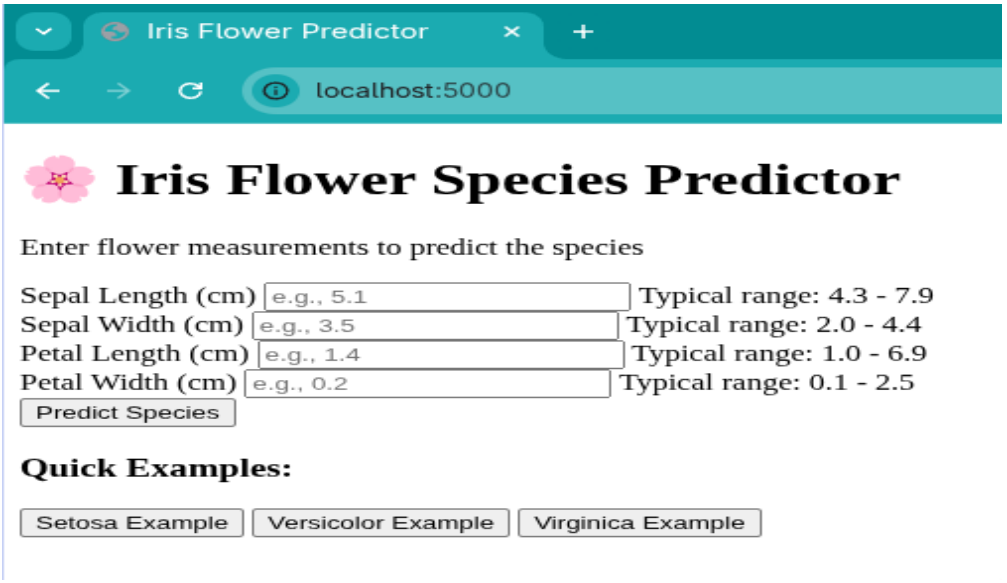
Output

An interactive web application with:

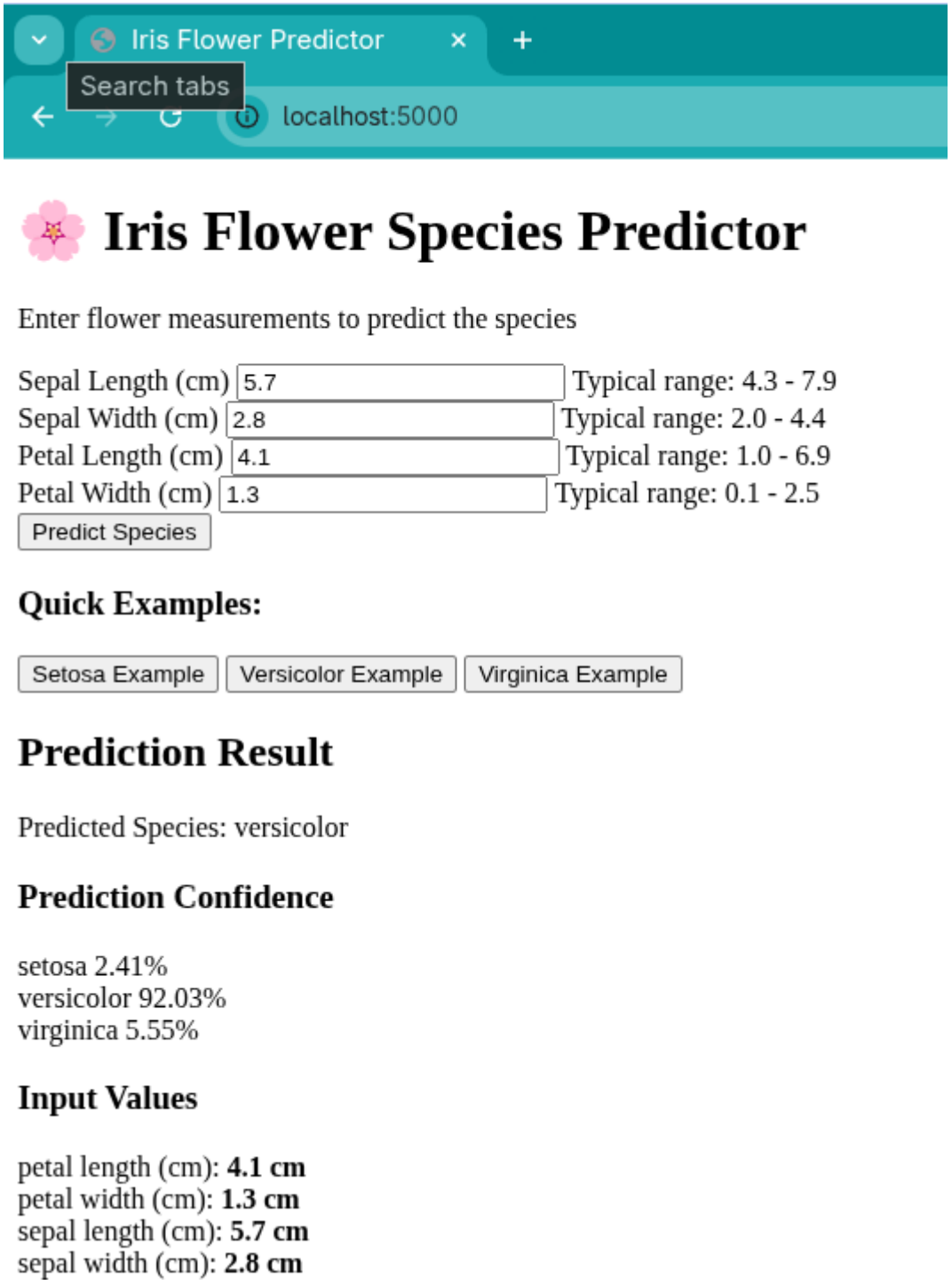
- Real-time ML predictions without page refresh
- Form validation and user-friendly error messages
- Loading indicators during API calls
- Animated prediction results with probability visualization
- Quick example buttons for testing
- Responsive design that works on mobile devices
- Smooth scrolling and fade-in animations

Screen-shots

Default page:



Prediction Result



Result

Successfully created an interactive frontend that communicates with ML API using JavaScript Fetch API, demonstrating modern asynchronous web development with real-time machine learning predictions.